UBUNTU LINUX

Terminal Commands

Comprehensive Student Practice Guide

Step-by-Step Instructions with Examples & Exercises

Topics Covered: File System Navigation • File Operations • Text Processing • Permissions • Processes • Networking • Package Management • Archiving • User Management • System Info • Scripting Basics

1. Getting Started with the Terminal

The terminal (also called the command line or shell) is a powerful text-based interface for controlling your Ubuntu system. Every command follows the same basic structure:

```
Syntax:  command  [options]  [arguments]
```

1.1 Opening the Terminal

Use any of these methods to open the Ubuntu terminal:

- Press Ctrl + Alt + T (keyboard shortcut)
- Right-click on the desktop → Open Terminal
- Search for 'Terminal' in the Applications menu
- Press the Super key and type 'terminal'

1.2 Understanding the Prompt

When you open a terminal, you will see something like:

```
username@hostname:~$
```

Where: username = your login name, hostname = computer name, ~ = home directory, $ = regular user (# means root/admin)

1.3 Essential Keyboard Shortcuts

Command	Description	Example
Ctrl + C	Stop / kill current command	Press when a command is stuck
Ctrl + Z	Pause/suspend a process	Send to background temporarily
Ctrl + D	Exit current session / EOF	Logout from terminal
Ctrl + L	Clear the screen	Same as typing 'clear'
Tab	Auto-complete commands/paths	Type 'doc' then press Tab
Up/Down Arrow	Browse command history	Navigate previous commands
Ctrl + A	Move cursor to line start	Jump to beginning of line
Ctrl + E	Move cursor to line end	Jump to end of line
Ctrl + R	Reverse search history	Search your past commands
Ctrl + Shift + C	Copy text in terminal	Copy selected text
Ctrl + Shift + V	Paste in terminal	Paste copied text

2. File System Navigation

The Linux file system is organized as a tree structure starting from / (root). Understanding how to navigate it is the most fundamental skill.

2.1 Basic Navigation Commands

Command	Description	Example
`pwd`	Print Working Directory — shows where you are	`pwd`
`ls`	List files and folders in current directory	`ls`
`ls -l`	List with detailed info (permissions, size, date)	`ls -l`
`ls -la`	List all files including hidden files	`ls -la`
`ls -lh`	List with human-readable file sizes	`ls -lh`
`ls -lt`	List sorted by modification time	`ls -lt`
`cd dirname`	Change directory (navigate into folder)	`cd Documents`
`cd ..`	Go up one directory level	`cd ..`
`cd ~`	Go to your home directory	`cd ~`
`cd /`	Go to root directory	`cd /`
`cd -`	Go back to previous directory	`cd -`
`tree`	Display directory structure as tree	`tree /home/user`

2.2 Step-by-Step Navigation Practice

Exercise 1: Explore Your System

Follow these commands in order:

```
pwd                    # See your current location
ls                     # List files here
cd /                   # Go to root
ls                     # See all top-level folders
cd /home               # Navigate to home directory
ls                     # List all user home folders
cd ~                   # Return to your home directory
pwd                    # Confirm you are back home
```

 Note: Important Linux directories: /home (user home folders), /etc (configuration files), /var (variable data/logs), /tmp (temporary files), /usr (user programs), /bin (essential binaries), /boot (boot files)

3. File and Directory Operations

Creating, copying, moving, and deleting files and directories are everyday operations. Practice these carefully — deletion in Linux is permanent by default.

3.1 Creating Files and Directories

Command	Description	Example
`touch filename`	Create an empty file	`touch notes.txt`
`touch f1 f2 f3`	Create multiple files at once	`touch a.txt b.txt c.txt`
`mkdir dirname`	Create a new directory	`mkdir Projects`
`mkdir -p path`	Create nested directories at once	`mkdir -p a/b/c`
`echo 'text' > file`	Create file with content	`echo 'Hello' > hello.txt`
`cat > filename`	Create file interactively (Ctrl+D to save)	`cat > myfile.txt`

3.2 Viewing File Contents

Command	Description	Example
`cat file`	Display entire file content	`cat readme.txt`
`cat -n file`	Display with line numbers	`cat -n script.sh`
`less file`	View large files page by page (q to quit)	`less /var/log/syslog`
`more file`	View file content with paging	`more bigfile.txt`
`head file`	Show first 10 lines of file	`head access.log`
`head -n 20 file`	Show first N lines	`head -n 20 file.txt`
`tail file`	Show last 10 lines of file	`tail error.log`
`tail -f file`	Live-monitor file (follow updates)	`tail -f /var/log/syslog`
`tail -n 50 file`	Show last N lines	`tail -n 50 log.txt`

3.3 Copying, Moving, and Deleting

Command	Description	Example
`cp source dest`	Copy a file to destination	`cp file.txt backup.txt`
`cp -r src/ dest/`	Copy entire directory recursively	`cp -r Projects/ Backup/`

Command	Description	Example
`cp -p src dest`	Copy preserving permissions/timestamps	`cp -p config.cfg /etc/`
`mv source dest`	Move or rename a file/directory	`mv old.txt new.txt`
`mv file /path/`	Move file to another directory	`mv report.pdf ~/Documents/`
`rm file`	Delete a file (permanent!)	`rm unwanted.txt`
`rm -i file`	Delete with confirmation prompt	`rm -i important.txt`
`rm -r directory`	Delete directory and all contents	`rm -r OldProject/`
`rm -rf directory`	Force delete without prompts (DANGER!)	`rm -rf /tmp/cache/`
`rmdir dirname`	Delete an empty directory only	`rmdir EmptyFolder`

 Tip: Always use 'rm -i' when you are unsure. The -i flag asks for confirmation before each deletion. Never use 'rm -rf' on important directories.

3.4 File Operations Practice

Exercise 2: Create a Practice Workspace

```
mkdir ~/practice                    # Create practice folder
cd ~/practice                       # Enter it
touch file1.txt file2.txt           # Create two files
mkdir subdir1 subdir2               # Create two subdirectories
echo 'Hello World' > file1.txt      # Add content to file1
cat file1.txt                       # Verify content
cp file1.txt subdir1/               # Copy file to subdir1
mv file2.txt subdir2/renamed.txt    # Move and rename file2
ls -la                              # List everything
ls subdir1/ subdir2/                # List subdirectories
```

4. Text Processing Commands

Linux provides powerful tools for searching, filtering, and transforming text. These are essential for working with files, logs, and data.

4.1 Searching with grep

Command	Description	Example
`grep 'word' file`	Search for word in file	`grep 'error' log.txt`
`grep -i 'word' file`	Case-insensitive search	`grep -i 'ERROR' log.txt`
`grep -n 'word' file`	Show line numbers with matches	`grep -n 'fail' app.log`
`grep -r 'word' dir/`	Search recursively in directory	`grep -r 'TODO' ~/code/`
`grep -v 'word' file`	Show lines NOT matching	`grep -v '#' config.txt`
`grep -c 'word' file`	Count matching lines	`grep -c 'error' log.txt`
`grep -l 'word' *.txt`	List files containing word	`grep -l 'import' *.py`
`grep -A 3 'word' file`	Show 3 lines after match	`grep -A 3 'error' log`
`grep -B 3 'word' file`	Show 3 lines before match	`grep -B 3 'error' log`

4.2 Text Manipulation

Command	Description	Example
`sort file`	Sort lines alphabetically	`sort names.txt`
`sort -n file`	Sort numerically	`sort -n numbers.txt`
`sort -r file`	Sort in reverse order	`sort -r list.txt`
`sort -u file`	Sort and remove duplicates	`sort -u data.txt`
`uniq file`	Remove consecutive duplicate lines	`uniq sorted.txt`
`wc file`	Count lines, words, characters	`wc essay.txt`
`wc -l file`	Count lines only	`wc -l script.py`
`wc -w file`	Count words only	`wc -w article.txt`
`cut -d: -f1 file`	Extract field 1 using : delimiter	`cut -d: -f1 /etc/passwd`
`awk '{print $1}' file`	Print first column of each line	`awk '{print $1}' data.txt`
`sed 's/old/new/g' file`	Replace all occurrences of 'old' with 'new'	`sed 's/hello/hi/g' f.txt`
`tr 'a-z' 'A-Z'`	Translate lowercase to uppercase	`echo 'hello' | tr 'a-z' 'A-Z'`
`diff file1 file2`	Show differences between two files	`diff original.txt edited.txt`

4.3 Pipes and Redirection

Pipes and redirection let you chain commands together and control where output goes.

Command	Description	Example
`cmd1 | cmd2`	Pipe output of cmd1 as input to cmd2	`ls -l | grep '.txt'`
`cmd > file`	Redirect output to file (overwrite)	`ls > filelist.txt`
`cmd >> file`	Append output to file	`date >> log.txt`
`cmd < file`	Use file as input to command	`sort < names.txt`
`cmd 2> file`	Redirect error output to file	`ls /x 2> errors.txt`
`cmd &> file`	Redirect both output and errors	`make &> build.log`
`cmd | tee file`	Output to both screen and file	`ls | tee list.txt`
`cmd1 && cmd2`	Run cmd2 only if cmd1 succeeds	`mkdir dir && cd dir`
`cmd1 || cmd2`	Run cmd2 only if cmd1 fails	`cd dir || mkdir dir`

 Tip: The pipe | is one of the most powerful features of Linux. It lets you combine simple commands to do complex tasks.

5. File Permissions

Linux uses a permission system to control who can read, write, or execute files. Understanding permissions is critical for security and system administration.

5.1 Understanding Permissions

When you run 'ls -l', you see permissions like: -rwxrw-r--

```
-  rwx  rw-  r--
^   ^    ^    ^
|   |    |    └── Others: read only
|   |    └─────── Group: read + write
|   └──────────── Owner: read + write + execute
└──────────────── File type (- = file, d = directory, l = link)
```

5.2 Permission Commands

Command	Description	Example
`ls -l`	View file permissions	`ls -l myfile.txt`
`chmod 755 file`	Set permissions using octal notation	`chmod 755 script.sh`
`chmod +x file`	Add execute permission	`chmod +x run.sh`
`chmod -x file`	Remove execute permission	`chmod -x run.sh`
`chmod +r file`	Add read permission	`chmod +r document.txt`
`chmod -w file`	Remove write permission (protect file)	`chmod -w important.txt`
`chmod u+x file`	Add execute for owner only	`chmod u+x myscript.sh`
`chmod go-w file`	Remove write from group and others	`chmod go-w config.conf`
`chmod -R 644 dir/`	Apply permissions recursively	`chmod -R 644 www/`
`chown user file`	Change file owner	`chown john file.txt`
`chown user:grp file`	Change owner and group	`chown john:dev script.sh`
`chgrp group file`	Change group ownership	`chgrp staff report.txt`

5.3 Octal Permission Reference

Octal	Permissions	Symbol	Common Use
777	Full access	`rwxrwxrwx`	Scripts (avoid for security)
755	Owner all, others read/exec	`rwxr-xr-x`	Directories, executables

Octal	Permissions	Symbol	Common Use
`644`	Owner rw, others read	`rw-r--r--`	Regular files, configs
`600`	Owner only rw	`rw-------`	Private keys, passwords
`700`	Owner only full	`rwx------`	Private scripts
`400`	Owner read only	`r--------`	Read-only files

6. Process Management

Processes are running programs. Linux provides powerful tools to monitor, control, and manage processes.

6.1 Viewing Processes

Command	Description	Example
`ps`	Show your current processes	`ps`
`ps aux`	Show all system processes in detail	`ps aux`
`ps aux | grep name`	Find a specific process	`ps aux | grep firefox`
`top`	Real-time interactive process viewer	`top`
`htop`	Enhanced interactive process viewer	`htop`
`pgrep name`	Get PID(s) of named process	`pgrep firefox`
`pidof name`	Get PID of running program	`pidof apache2`

6.2 Controlling Processes

Command	Description	Example
`kill PID`	Terminate process by PID	`kill 1234`
`kill -9 PID`	Force-kill process (SIGKILL)	`kill -9 5678`
`killall name`	Kill all processes by name	`killall firefox`
`pkill name`	Kill processes matching name pattern	`pkill -f myapp`
`command &`	Run command in background	`sleep 100 &`
`jobs`	List background jobs in current session	`jobs`
`fg %1`	Bring job 1 to foreground	`fg %1`
`bg %1`	Resume job 1 in background	`bg %1`
`nohup cmd &`	Run command immune to hangup	`nohup ./server.sh &`
`nice -n 10 cmd`	Run command with lower priority	`nice -n 10 make`
`renice 5 PID`	Change running process priority	`renice 5 1234`

6.3 System Monitoring

Command	Description	Example
`free -h`	Show RAM usage (human readable)	`free -h`
`df -h`	Show disk space usage	`df -h`
`df -h /home`	Show disk space for specific mount	`df -h /home`
`du -sh dir/`	Show directory total size	`du -sh ~/Documents/`
`du -h --max-depth=1`	Show sizes of subdirectories	`du -h --max-depth=1 /var/`
`uptime`	Show system uptime and load	`uptime`
`vmstat`	Virtual memory statistics	`vmstat 2 5`
`iostat`	CPU and I/O statistics	`iostat -x 2`
`lscpu`	Display CPU information	`lscpu`
`lsblk`	List block devices (disks)	`lsblk`

7. Package Management (APT)

Ubuntu uses APT (Advanced Package Tool) to install, update, and remove software. Always run with sudo (administrator privileges).

7.1 APT Commands

Command	Description	Example
`sudo apt update`	Refresh package list from repositories	`sudo apt update`
`sudo apt upgrade`	Upgrade all installed packages	`sudo apt upgrade`
`sudo apt full-upgrade`	Upgrade with dependency changes	`sudo apt full-upgrade`
`sudo apt install pkg`	Install a package	`sudo apt install vim`
`sudo apt install p1 p2`	Install multiple packages	`sudo apt install git curl wget`
`sudo apt remove pkg`	Remove package (keep config)	`sudo apt remove firefox`
`sudo apt purge pkg`	Remove package and config files	`sudo apt purge apache2`
`sudo apt autoremove`	Remove unused dependency packages	`sudo apt autoremove`
`sudo apt autoclean`	Clean downloaded package files	`sudo apt autoclean`
`apt search keyword`	Search for packages	`apt search text-editor`
`apt show pkg`	Show package details	`apt show nginx`
`apt list --installed`	List all installed packages	`apt list --installed`
`dpkg -l`	List installed packages (dpkg)	`dpkg -l | grep python`
`dpkg -i file.deb`	Install .deb file directly	`sudo dpkg -i chrome.deb`

7.2 Common Useful Packages for Students

Command	Description	Example
`git`	Version control system	`sudo apt install git`
`vim / nano`	Text editors	`sudo apt install vim nano`
`curl / wget`	Download files from web	`sudo apt install curl wget`
`python3-pip`	Python package manager	`sudo apt install python3-pip`
`build-essential`	Compilers & build tools (gcc, make)	`sudo apt install build-essential`
`net-tools`	Network tools (ifconfig, netstat)	`sudo apt install net-tools`

Command	Description	Example
`tree`	Display directory tree	`sudo apt install tree`
`htop`	Interactive process viewer	`sudo apt install htop`
`tmux`	Terminal multiplexer	`sudo apt install tmux`
`zip / unzip`	Archive tools	`sudo apt install zip unzip`

8. Networking Commands

These commands help you view network configuration, test connectivity, transfer files, and manage network connections.

8.1 Network Information

Command	Description	Example
`ip addr`	Show IP addresses of all interfaces	`ip addr`
`ip addr show eth0`	Show info for specific interface	`ip addr show enp3s0`
`ifconfig`	Show network interfaces (older)	`ifconfig`
`hostname`	Show system hostname	`hostname`
`hostname -I`	Show all IP addresses	`hostname -I`
`ip route`	Show routing table	`ip route`
`cat /etc/resolv.conf`	Show DNS servers	`cat /etc/resolv.conf`
`nslookup domain`	Look up DNS for domain	`nslookup google.com`
`dig domain`	Detailed DNS query	`dig ubuntu.com`

8.2 Testing Connectivity

Command	Description	Example
`ping host`	Test connectivity (Ctrl+C to stop)	`ping google.com`
`ping -c 4 host`	Send only 4 ping packets	`ping -c 4 8.8.8.8`
`traceroute host`	Trace network path to host	`traceroute google.com`
`curl url`	Fetch web page / test HTTP	`curl https://example.com`
`curl -I url`	Show only HTTP response headers	`curl -I google.com`
`wget url`	Download file from URL	`wget https://example.com/file.zip`
`wget -O name url`	Download and save with custom name	`wget -O ubuntu.iso url`
`nc -zv host port`	Test if port is open	`nc -zv google.com 443`
`ss -tulpn`	Show listening ports and services	`ss -tulpn`
`netstat -tulpn`	Show network connections	`netstat -tulpn`

8.3 SSH — Remote Access

Command	Description	Example
`ssh user@host`	Connect to remote server via SSH	`ssh john@192.168.1.10`
`ssh -p 2222 user@host`	Connect to non-default SSH port	`ssh -p 2222 user@host`
`ssh-keygen`	Generate SSH key pair	`ssh-keygen -t rsa -b 4096`
`ssh-copy-id user@host`	Copy public key to server	`ssh-copy-id john@server`
`scp file user@host:path`	Copy file to remote server	`scp report.pdf user@server:~/`
`scp user@host:file .`	Download file from remote server	`scp user@host:~/data.csv .`
`scp -r dir/ user@host:`	Copy directory to remote	`scp -r project/ user@host:~/`
`exit`	Close SSH connection	`exit`

9. Archiving and Compression

Archiving lets you bundle files together. Compression reduces their size. Linux supports many formats.

9.1 tar — The Standard Archiver

Command	Description	Example
`tar -cvf arch.tar dir/`	Create a tar archive	`tar -cvf backup.tar ~/docs/`
`tar -xvf arch.tar`	Extract a tar archive	`tar -xvf backup.tar`
`tar -cvzf arch.tar.gz dir/`	Create compressed (gzip) archive	`tar -cvzf proj.tar.gz project/`
`tar -xvzf arch.tar.gz`	Extract gzip-compressed archive	`tar -xvzf proj.tar.gz`
`tar -cvjf arch.tar.bz2 dir/`	Create bzip2-compressed archive	`tar -cvjf arch.tar.bz2 dir/`
`tar -xvjf arch.tar.bz2`	Extract bzip2 archive	`tar -xvjf arch.tar.bz2`
`tar -tvf archive.tar`	List contents without extracting	`tar -tvf backup.tar`
`tar -xvzf arch.tar.gz -C /path/`	Extract to specific directory	`tar -xvzf f.tar.gz -C /tmp/`

9.2 zip, gzip, and bzip2

Command	Description	Example
`zip archive.zip files`	Create zip archive	`zip report.zip *.pdf`
`zip -r archive.zip dir/`	Zip entire directory	`zip -r website.zip www/`
`unzip archive.zip`	Extract zip archive	`unzip report.zip`
`unzip -l archive.zip`	List zip contents	`unzip -l project.zip`
`gzip file`	Compress file with gzip	`gzip large.log`
`gzip -d file.gz`	Decompress gzip file	`gzip -d large.log.gz`
`gunzip file.gz`	Same as gzip -d	`gunzip data.gz`
`bzip2 file`	Compress with bzip2 (better ratio)	`bzip2 bigfile.txt`
`bunzip2 file.bz2`	Decompress bzip2 file	`bunzip2 data.bz2`

 Tip: tar flags memory aid: c=create, x=extract, v=verbose, f=filename, z=gzip, j=bzip2, t=list. Example: tar -cvzf = Create Verbose gZip File

10. User and Group Management

Linux is a multi-user system. These commands let you manage users, groups, and switch between accounts.

10.1 User Management

Command	Description	Example
`whoami`	Show currently logged-in user	`whoami`
`id`	Show user ID, group ID, and groups	`id`
`id username`	Show info for a specific user	`id john`
`who`	List all logged-in users	`who`
`w`	Show logged-in users and activity	`w`
`last`	Show login history	`last`
`sudo useradd username`	Create a new user account	`sudo useradd alice`
`sudo useradd -m -s /bin/bash user`	Create user with home dir and shell	`sudo useradd -m alice`
`sudo passwd username`	Set or change user password	`sudo passwd alice`
`sudo usermod -aG group user`	Add user to a group	`sudo usermod -aG sudo alice`
`sudo userdel username`	Delete user account	`sudo userdel alice`
`sudo userdel -r username`	Delete user and home directory	`sudo userdel -r alice`

10.2 Switching Users and sudo

Command	Description	Example
`sudo command`	Run command as administrator	`sudo apt update`
`sudo -i`	Open root shell session	`sudo -i`
`su username`	Switch to another user	`su alice`
`su -`	Switch to root user	`su -`
`exit`	Return to original user	`exit`
`sudo visudo`	Edit sudoers file safely	`sudo visudo`
`groups`	Show your group memberships	`groups`
`groups username`	Show groups for specific user	`groups alice`
`sudo groupadd name`	Create a new group	`sudo groupadd developers`
`sudo gpasswd -a user grp`	Add user to group	`sudo gpasswd -a alice developers`

 Note: sudo stands for 'superuser do'. It lets regular users run administrative commands. Be careful — mistakes with sudo can damage your system.

11. System Information Commands

Command	Description	Example
`uname -a`	Show all system information	`uname -a`
`uname -r`	Show kernel version only	`uname -r`
`lsb_release -a`	Show Ubuntu version details	`lsb_release -a`
`cat /etc/os-release`	Show OS release information	`cat /etc/os-release`
`hostname`	Show system hostname	`hostname`
`date`	Show current date and time	`date`
`date '+%Y-%m-%d %H:%M'`	Show formatted date	`date '+%Y-%m-%d'`
`cal`	Display calendar	`cal`
`uptime`	System uptime and load averages	`uptime`
`history`	Show command history	`history`
`history | grep cmd`	Search command history	`history | grep apt`
`!123`	Re-run command number 123	`!123`
`!!`	Re-run the last command	`!!`
`env`	Show all environment variables	`env`
`echo $HOME`	Show value of HOME variable	`echo $PATH`
`export VAR=value`	Set environment variable	`export EDITOR=vim`
`alias ll='ls -la'`	Create command alias	`alias ll='ls -la'`
`which command`	Show location of command	`which python3`
`whereis command`	Find binary, source, man pages	`whereis gcc`
`man command`	Show manual page for command	`man ls`
`command --help`	Show quick help for command	`ls --help`

12. Finding Files

Linux provides several powerful tools to locate files anywhere on the system.

Command	Description	Example
`find /path -name 'file'`	Find file by name	`find / -name 'config.txt'`
`find . -name '*.py'`	Find all Python files here	`find . -name '*.py'`
`find . -type f`	Find only regular files	`find . -type f`
`find . -type d`	Find only directories	`find . -type d`
`find . -size +10M`	Find files larger than 10MB	`find /var -size +10M`
`find . -mtime -7`	Files modified in last 7 days	`find . -mtime -7`
`find . -user john`	Files owned by user john	`find / -user john`
`find . -perm 644`	Find files with permission 644	`find . -perm 644`
`find . -name '*.log' -delete`	Find and delete files	`find /tmp -name '*.tmp' -delete`
`find . -exec cmd {} \;`	Execute command on found files	`find . -name '*.txt' -exec wc -l {} \;`
`locate filename`	Fast file search (uses database)	`locate ubuntu.iso`
`updatedb`	Update the locate database	`sudo updatedb`
`which cmd`	Find where a command is located	`which python3`
`type cmd`	Show how command is interpreted	`type ls`

13. Shell Scripting Basics

Shell scripts let you automate repetitive tasks by combining commands in a file. Scripts use the .sh extension.

13.1 Your First Script

Step 1: Create the script file

```
nano myscript.sh
```

Step 2: Write the script content

```
#!/bin/bash
# This is a comment
echo 'Hello, World!'
echo 'Today is: ' $(date)
echo 'You are logged in as: ' $(whoami)
```

Step 3: Save and make executable

```
chmod +x myscript.sh
```

Step 4: Run the script

```
./myscript.sh
```

13.2 Script with Variables

```
#!/bin/bash
NAME='Alice'
AGE=20
echo "Hello, $NAME!"
echo "You are $AGE years old."
echo "In 5 years, you will be $((AGE + 5)) years old."
```

13.3 Script with User Input

```
#!/bin/bash
echo 'What is your name?'
read NAME
echo "Hello, $NAME! Welcome to Linux."
```

13.4 Script with if-else

```
#!/bin/bash
read -p 'Enter a number: ' NUM
if [ $NUM -gt 10 ]; then
    echo 'Number is greater than 10'
elif [ $NUM -eq 10 ]; then
```

```
    echo 'Number is exactly 10'
else
    echo 'Number is less than 10'
fi
```

13.5 Script with for loop

```
#!/bin/bash
for i in 1 2 3 4 5; do
    echo "Iteration number: $i"
done
```

```
# Loop through files
for file in *.txt; do
    echo "Processing: $file"
    wc -l "$file"
done
```

13.6 Script with while loop

```
#!/bin/bash
COUNT=1
while [ $COUNT -le 5 ]; do
    echo "Count: $COUNT"
    COUNT=$((COUNT + 1))
done
```

13.7 Script with Functions

```
#!/bin/bash
greet() {
    echo "Hello, $1!"
}
```

```
backup_files() {
    tar -czf backup_$(date +%Y%m%d).tar.gz $1
    echo "Backup created successfully!"
}
```

```
greet 'World'
greet 'Student'
backup_files ~/Documents
```

 Tip: Always start scripts with #!/bin/bash (called a shebang). This tells the system which interpreter to use to run the script.

14. Comprehensive Practice Exercises

Exercise A: File System Exploration

Complete these tasks in order:

```
# 1. Go to root and explore
cd / && ls -la
# 2. Explore system directories
ls /etc | head -20
ls /var/log | head -10
# 3. Return home and create workspace
cd ~ && mkdir -p lab/files lab/scripts lab/backup
# 4. Verify structure
tree ~/lab
```

Exercise B: Text Processing Pipeline

```
# 1. Create test data
echo -e 'Alice\nBob\nCharlie\nAlice\nDave\nBob' > names.txt
# 2. Sort and find unique names
sort names.txt | uniq
# 3. Count occurrences
sort names.txt | uniq -c | sort -rn
# 4. Search and redirect
grep 'A' names.txt > a_names.txt && cat a_names.txt
```

Exercise C: Permission Management

```
# 1. Create files with different permissions
touch public.txt private.txt executable.sh
# 2. Set permissions
chmod 644 public.txt
chmod 600 private.txt
chmod 755 executable.sh
# 3. Verify permissions
ls -la public.txt private.txt executable.sh
# 4. Add content and test
echo 'echo Hello from script' > executable.sh
./executable.sh
```

Exercise D: Process Management

```
# 1. Start background processes
sleep 300 &
sleep 400 &
# 2. View jobs
jobs
# 3. Find PIDs
```

```
pgrep sleep
# 4. View in top (press q to exit)
top
# 5. Kill the processes
killall sleep
jobs
```

Exercise E: Complete Mini-Project — System Report Script

Create a script that generates a system information report:

```
#!/bin/bash
REPORT_FILE="system_report_$(date +%Y%m%d_%H%M%S).txt"

echo '================================' > $REPORT_FILE
echo '      SYSTEM INFORMATION REPORT ' >> $REPORT_FILE
echo '================================' >> $REPORT_FILE
echo "Date: $(date)" >> $REPORT_FILE
echo "Hostname: $(hostname)" >> $REPORT_FILE
echo "User: $(whoami)" >> $REPORT_FILE
echo '' >> $REPORT_FILE
echo '--- OS Information ---' >> $REPORT_FILE
lsb_release -d >> $REPORT_FILE
uname -r >> $REPORT_FILE
echo '' >> $REPORT_FILE
echo '--- Memory Usage ---' >> $REPORT_FILE
free -h >> $REPORT_FILE
echo '' >> $REPORT_FILE
echo '--- Disk Usage ---' >> $REPORT_FILE
df -h >> $REPORT_FILE
echo '' >> $REPORT_FILE
echo '--- Top 5 Processes (CPU) ---' >> $REPORT_FILE
ps aux --sort=-%cpu | head -6 >> $REPORT_FILE

echo "Report saved to: $REPORT_FILE"
cat $REPORT_FILE
```

15. Quick Reference Cheat Sheet

Navigation & Files	System & Admin
`pwd, ls, cd, tree`	`whoami, id, sudo, su`
`touch, mkdir, cat, less`	`ps aux, top, htop, kill`
`cp, mv, rm, rmdir`	`free -h, df -h, du -sh`
`grep, sort, uniq, wc`	`apt update, apt install`
`cut, awk, sed, tr`	`chmod, chown, chgrp`
`find, locate, which`	`ssh, scp, ping, curl`
`tar, zip, gzip, unzip`	`ifconfig, ip addr, ss`
`| pipe, >, >>, <`	`uname, lsb_release, date`

Remember: Linux is learned by doing. Practice each command, make mistakes, and learn from them. The terminal is your most powerful tool as a developer or system administrator. Keep exploring!